
Homework 1: Final Submission

IE 598: Reinforcement Learning and Learning-Based Control

Dynamic programming, partial observation, and convex interpolation

Prachit Gupta

NetID: prachit2

chatgpt cli was used to generate this report from my handwritten solutions for Q1 and Q3 . The entire repo containing the handwritten solutions and md files used for generating reports for Q2 are available for review at [link](#)

September 18, 2026

Contents

1	Question 1	1
2	Question 2	3
2.1	1(a)	3
2.2	1(b)	3
2.3	1(c)	3
2.4	2(a)	4
2.5	2(b)	4
2.6	3	5
2.7	4	5
2.8	5	5
3	Question 3	6
(a)		6
(b)		6
(c)		7
(d)		7

1 Question 1

Given:

$$w_1 = \begin{cases} 1 & \text{Prob } 1/2 \\ 3 & \text{Prob } 1/2 \end{cases} \quad c(x) = \begin{cases} x & \text{if } x \geq 0 \\ -1000x & \text{if } x < 0 \end{cases}$$

$$g_t(x_t, u_t) = c(x_t) + 2u_t, \quad g_N(x_N) = c(x_N)$$

let's start DP with terminal cost

at $t = N = 2$

$$g_2(x_2) = c(x_2) = \begin{cases} x_2 & x_2 \geq 0 \\ -1000x_2 & x_2 < 0 \end{cases} \quad J_2^*(x_2) = g_2(x_2)$$

at $t = N - 1$

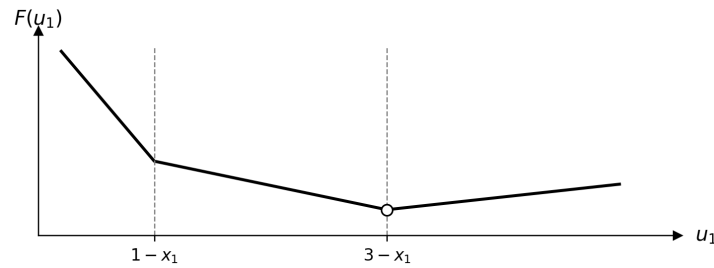
using DP

$$\begin{aligned} J_1^*(x_1) &= \mathbb{E}_{w_1} [c(x_1) + 2u_1 + J_2^*(x_2) \mid x_1, u_1] \\ &= c(x_1) + 2u_1 + \mathbb{E}_{w_1} [J_2^*(x_1 + u_1 - w_1)] \\ &= c(x_1) + 2u_1 + \frac{1}{2}J_2^*(x_1 + u_1 - 1) + \frac{1}{2}J_2^*(x_1 + u_1 - 3) \end{aligned}$$

let

$$F(u_1) = \begin{cases} 2u_1 + \frac{1}{2}[-1000(x_1 + u_1 - 1)] + \frac{1}{2}[-1000(x_1 + u_1 - 3)] & x_1 + u_1 < 1 \\ 2u_1 + \frac{1}{2}(x_1 + u_1 - 1) + \frac{1}{2}[-1000(x_1 + u_1 - 3)] & 1 \leq x_1 + u_1 < 3 \\ 2u_1 + \frac{1}{2}(x_1 + u_1 - 1) + \frac{1}{2}(x_1 + u_1 - 3) & x_1 + u_1 \geq 3 \end{cases}$$

$$F(u_1) = \begin{cases} -998u_1 - 1000x_1 + 2000 & u_1 < 1 - x_1 \\ -\frac{995u_1}{2} - \frac{999x_1}{2} + \frac{2999}{2} & 1 - x_1 \leq u_1 < 3 - x_1 \\ 3u_1 + x_1 - 2 & u_1 \geq 3 - x_1 \end{cases}$$



$$F(u_1) = \text{minimum at } u_1^* = 3 - x_1$$

$$\boxed{J_1^*(x_1) = c(x_1) + 7 - 2x_1 \quad \text{at } u_1^* = 3 - x_1}$$

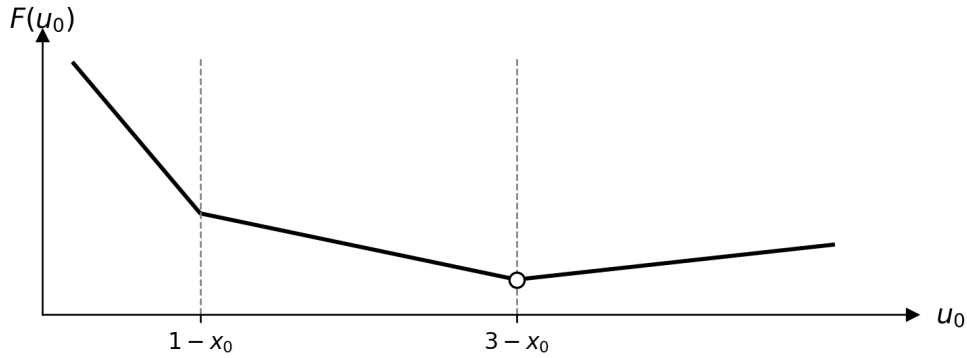
now

$$\begin{aligned}
 J_0(x_0) &= \mathbb{E}_{w_0} [c(x_0) + 2u_0 + J_1^*(x_1) \mid x_0, u_0] \\
 &= c(x_0) + 2u_0 + \mathbb{E}_{w_0} [J_1^*(x_0 + u_0 - w_0)] \\
 &= c(x_0) + 2u_0 + \underbrace{\frac{1}{2}J_1^*(x_0 + u_0 - 1) + \frac{1}{2}J_1^*(x_0 + u_0 - 3)}_{F(u_0)}
 \end{aligned}$$

$$J_1^*(x_1) = \begin{cases} -1000x_1 + 7 - 2x_1 & x_1 < 0 \\ 7 - x_1 & x_1 \geq 0 \end{cases}$$

$$F(u_0) = \begin{cases} 2u_0 + \frac{-1002(x_0 + u_0 - 1) + 7}{2} + \frac{-1002(x_0 + u_0 - 3) + 7}{2} & x_0 + u_0 < 1 \\ 2u_0 + \frac{7 - (x_0 + u_0 - 1)}{2} + \frac{-1002(x_0 + u_0 - 3) + 7}{2} & 1 \leq x_0 + u_0 < 3 \\ 2u_0 + \frac{7 - (x_0 + u_0 - 1)}{2} + \frac{7 - (x_0 + u_0 - 3)}{2} & x_0 + u_0 \geq 3 \end{cases}$$

$$F(u_0) = \begin{cases} -1000u_0 - 1002x_0 + 2011 & u_0 < 1 - x_0 \\ -\frac{999u_0}{2} - \frac{1003x_0}{2} + \frac{3021}{2} & 1 - x_0 \leq u_0 < 3 - x_0 \\ u_0 - x_0 + 9 & u_0 \geq 3 - x_0 \end{cases}$$



optimum value occurs at

$$u_0^* = 3 - x_0$$

$$F(u_0^*) = 12 - 2x_0$$

$$\boxed{J_0^*(x_0) = c(x_0) + 12 - 2x_0}$$

$$J_0^*(0) = 12$$

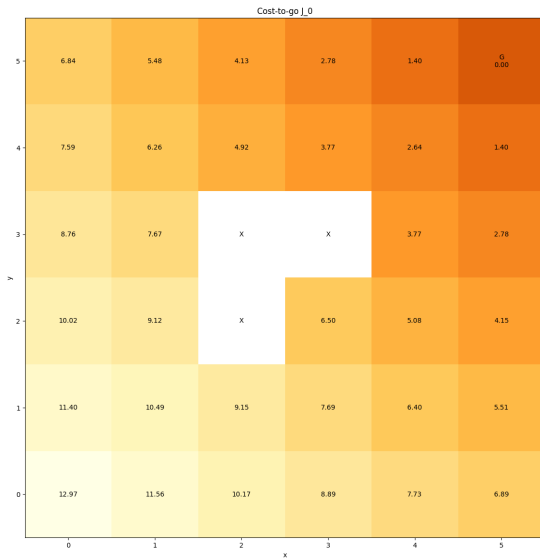
2 Question 2

2.1 1(a)

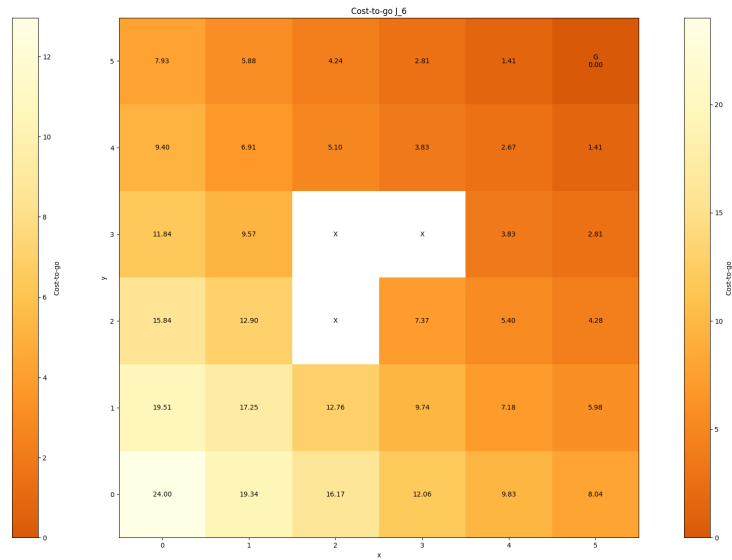
MDP.py

```
J_opt[t,s_index] = min([stage_cost_c(s_index, a) + sum([transition_prob[a, s_index, s_next] *
J_opt[t + 1, s_next] for s_next in range(S)]) for a in range(A)])
```

2.2 1(b)



(a) J_0^*



(b) J_6^*

2.3 1(c)

As evident in the heat maps, For the same state (x), ($J_6^*(x)$) is generally larger than ($J_0^*(x)$). At ($t=6$), the robot has fewer remaining time steps before the terminal horizon, so it has fewer opportunities to reach the goal. Hence, it has a higher probability of remaining outside the goal at ($t=N$) and incurring the terminal penalty of 15. In contrast, starting from ($t=0$) gives the robot more time to reach the absorbing goal, after which the stage cost is zero.

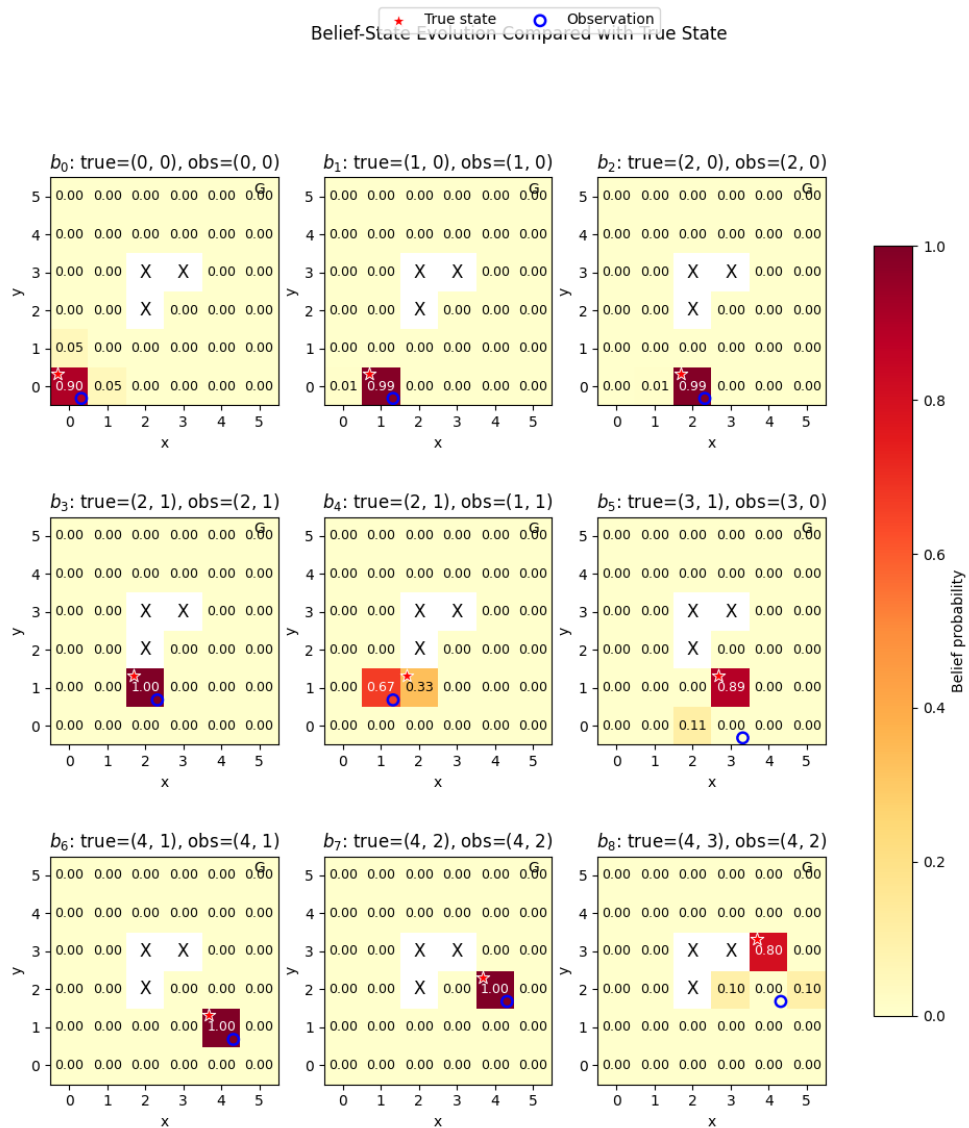
Although the Bellman equation adds the current stage cost to the expected future cost, this does not imply that the cost-to-go must increase as we move backward in time which was my initial intuition. The future-cost term also changes because an earlier starting time provides more opportunities to reach the goal and avoid the relatively large terminal penalty. In this case for example, that benefit outweighs the extra stage costs, so ($J_0^*(x)$) is generally smaller than ($J_6^*(x)$).

2.4 2(a)

POMDP.py

```
term1[s_next] += belief[s] * transition_prob[action_index, s, s_next] * observation_prob[
    s_next, observation_index]
```

2.5 2(b)



2.6 3

MAP.py

```
best_action = np.argmin([stage_cost_c(most_likely_state, a) + sum([transition_prob[a,
    most_likely_state, s_next] * J_opt[t + 1, s_next] for s_next in range(S)]) for a in range(
    A)])
```

2.7 4

The QMDP policy uses the entire belief distribution rather than only the most likely state. For each action, it computes the expected MDP cost by weighting the cost associated with each possible current state by its belief probability, and then chooses the action with the smallest expected cost. Intuitively, it accounts for uncertainty in the robot's location instead of committing to a single estimated state.

2.8 5

Policy	Success Rate	Average Cost	Avg. Time to Goal
MAP	0.925	13.085	11.714
QMDP	0.941	12.774	11.694

intuitive explanation

QMDP accounts for uncertainty in the existing state rather than assuming the robot to be at the most likely state.

The tradeoff is that you may miss out on those actions which are more advantageous at the most likely state but poor or incur a high penalty if the robot is actually in another state. The nature of the belief distribution also matters. If the second most likely state is not that much lower in probability than the most likely state, not accounting for the case when the robot is actually in that state can lead to failure runs.

Intuitively, I feel that MAP should lead to faster goal reaching since it assumes the most likely state to be the only state for which it has to compute the minimum over all actions. However, as evident in the results, the more substantial treatment of state uncertainty in QMDP, at least for our example, does not appear to incur more time to reach the goal. This may not necessarily remain the case as the dimensions of the state and action spaces scale up, but that cannot be concluded from this experiment alone.

Therefore, QMDP performs better in all three reported metrics for this experiment: higher success rate, lower average cost, and slightly lower average time-to-goal. However, the difference in average time-to-goal is very small, so it would be better not to make a strong deterministic conclusion that QMDP is inherently faster than MAP.

3 Question 3

(a)

$\hat{S}$ is finite subset of its 'convex' hull S

$$\Rightarrow \text{given } \sum_{i=1}^M \lambda_i s_i = x \quad \text{where } \sum \lambda_i = 1$$

$$\tilde{J}_k(x) = \tilde{J}_k\left(\sum_{i=1}^M \lambda_i s_i\right)$$

using convexity (Jensen's inequality)

$$\hat{J}_k(x) = \hat{J}_k\left(\sum_{i=1}^M \lambda_i s_i\right) \leq \sum_{i=1}^M \lambda_i \hat{J}_k(s_i)$$

$\therefore$ this is true for every feasible λ that includes minimizing one

$$\begin{aligned} \Rightarrow \hat{J}_k(x) &\leq \min \left\{ \sum_{i=1}^M \lambda_i \hat{J}_k(s_i) \mid \lambda_i \in \text{feasible set} \right\} \\ &\Rightarrow \boxed{\hat{J}_k(x) \leq \tilde{J}_k(x)} \end{aligned}$$

(b)

using eq (2)

$$J_{N-1}(x) = \mathbb{E} [g_{N-1}(x, u_{N-1}(x), w_{N-1}) + J_N(f_{N-1}(x, u_{N-1}, w_{N-1}))]$$

where $u_{N-1}(x) = \vec{u}$ is policy optimizer

given

$$J_N(f_{N-1}(\cdot)) = \hat{J}_N(\cdot) = \tilde{J}_N(\cdot)$$

plugging this in

$$J_{N-1}(x) = \mathbb{E} [g_{N-1}(x, u_{N-1}(x), w_{N-1}) + \tilde{J}_N(f_{N-1}(x, u_{N-1}, w_{N-1}))]$$

$u_{N-1} = \vec{u}$ policy optimizing seq

$$J_{N-1}(x) = \min_u \mathbb{E} [g_{N-1}(x, u, w_{N-1}) + \tilde{J}_N(f_{N-1}(x, u, w_{N-1}))]$$

this is how $\hat{J}_k$ is defined

$$\text{so } J_{N-1}(x) = \hat{J}_{N-1}(x)$$

using Convexity

$$\hat{J}_{N-1}(x) \leq \tilde{J}_{N-1}(x) \quad (\text{proved in (a)})$$

$$\text{so } \boxed{J_{N-1}(x) = \hat{J}_{N-1}(x) \leq \tilde{J}_{N-1}(x)}$$

(c)

$$\begin{aligned}
J_{N-2}(x) &= \mathbb{E} [g_{N-2}(x, \vec{u}_{N-2}(x), w_{N-2}) + J_{N-1}(f_{N-2}(x, \vec{u}_{N-2}(x), w_{N-2}))] \\
&\leq \mathbb{E} [g_{N-2}(x, \vec{u}_{N-2}, w_{N-2}) + \tilde{J}_{N-1}(f_{N-2}(x, \vec{u}_{N-2}, w_{N-2}))]
\end{aligned}$$

(from part b)

 $\because \vec{u}_{N-2}$ is optimizing seq.

$$\begin{aligned}
&\leq \mathbb{E} [g_{N-2}(\cdot) + \tilde{J}_{N-1}(f_{N-2}(\cdot))] \\
&= \min_u \mathbb{E} [g_{N-2}(x, u, w_{N-2}) + \tilde{J}_{N-1}(f_{N-2}(x, u, w_{N-2}))] \\
&= \hat{J}_{N-2}(x)
\end{aligned}$$

definition of $\hat{J}_{N-2}$

& Convexity implies

$$\begin{aligned}
&\hat{J}_{N-2}(x) \leq \tilde{J}_{N-2}(x) \\
\Rightarrow &\boxed{J_{N-2}(x) \leq \hat{J}_{N-2}(x) \leq \tilde{J}_{N-2}(x)}
\end{aligned}$$

(d)

Proof by induction like we did in lecture 5

Base case $k = N$ ($\because$ we go backwards)

given

$$J_N(x) = \tilde{J}_N(x) = \hat{J}_N(x)$$

which is same as

$$\Rightarrow J_N(x) \leq \tilde{J}_N(x) \leq \hat{J}_N(x) \quad \text{so true } \checkmark$$

let's assume its true for $k+1$

$$\Rightarrow J_{k+1}(x) \leq \hat{J}_{k+1}(x) \leq \tilde{J}_{k+1}(x) \quad (2)$$

Using definition

$$\begin{aligned}
J_k(x_k) &= \mathbb{E} [g_k(x_k, \vec{u}_k, w_k) + J_{k+1}(f_k(x_k, \vec{u}_k, w_k))] \\
&\leq \mathbb{E} [g_k(x_k, \vec{u}_k, w_k) + \tilde{J}_{k+1}(f_k(x_k, \vec{u}_k, w_k))] \quad (\text{using (2)}) \\
&\leq \min_{u_k \in U_k} \mathbb{E} [g_k(x_k, u_k, w_k) + \tilde{J}_{k+1}(f_k(x_k, u_k, w_k))] \\
&\leq \hat{J}_k(x_k) \quad (\text{using definition of } \hat{J})
\end{aligned}$$

also using Convexity

$$\hat{J}_k(x_k) \leq \tilde{J}_k(x_k)$$

so by induction

 $\because$ Base case is trivially True& k th case is true iff $k+1$ th case is True

We can say that

$$\boxed{J_k(x) \leq \hat{J}_k(x) \leq \tilde{J}_k(x)} \quad \forall x \in S \text{ \& } \forall k \in \{0, 1, 2, \dots, N\}$$